

Arquitectura técnica de cuatro sistemas

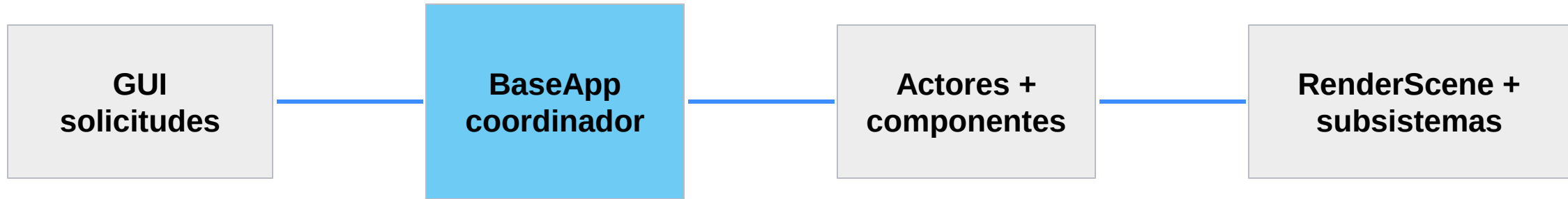
Play/Stop · audio espacial · partículas · culling y Octree

El alcance cubre cuatro áreas distintas del motor

Herramienta	Qué enseña	Criterio de aceptación (DoD)	Dificultad
Modo Play/Stop + scripting de comportamiento	Separar estado edición vs runtime; bind de scripting (Lua/C#) o componentes de comportamiento	Botón Play arranca lógica (rotar, mover); Stop restaura el estado guardado de la escena	★★★
Integración de física (Jolt / Bullet / PhysX, o rígida simple)	Rigidbody, colliders, step fijo, sincronizar transform↔física	RigidbodyComponent + colliders; caen y colisionan en Play	★★★★
Sistema de audio (DirectX/TK ya está disponible — Audio.h)	Audio 3D posicional, atenuación	AudioSourceComponent + listener en cámara; sonido espacial al moverse	★★★
Post-proceso sobre el deferred	Full-screen passes, HDR	Stack con Bloom + Tonemapping + FXAA/SSAO toggables; encaja tras el lighting pass que ya tienen	★★★
Sistema de partículas	Emisores, simulación CPU/GPU, billboard	ParticleEmitterComponent con curvas de vida/color; render aditivo	★★★
Animación esquelética (el FBX SDK ya está en el repo)	Skinning, jerarquía de huesos, blending	Importar malla con skeleton, AnimatorComponent reproduce clips	★★★★
Frustum culling + Octree + profiler	Particionado espacial, optimización	El profiler muestra objetos culled vs dibujados; mejora medible de frame-time	★★★
Material node editor (grafo visual)	Compilación/generación de shaders, AST de nodos	Conectar nodos genera el HLSL del material	★★★★

- Runtime: Play/Stop y comportamiento
- Sonido: fuente y listener espacial
- VFX: emisión, simulación y billboards
- Optimización: frustum, Octree y métricas

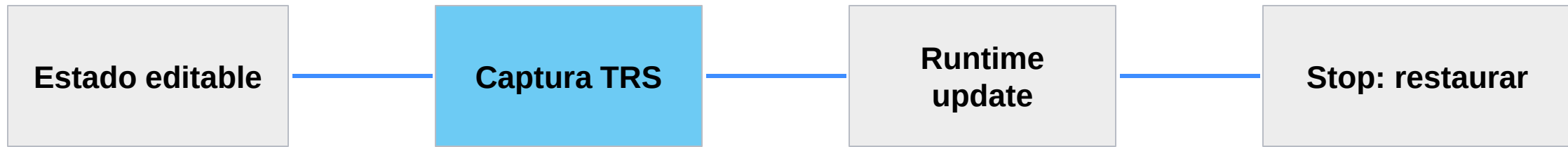
BaseApp coordina datos, actualización y render



La GUI publica intenciones. BaseApp conserva la propiedad de Device, escena, cámara, audio y pipeline.

```
update(deltaTime) -> culling -> render -> GUI -> Present
```

Play/Stop separa edición y ejecución



```
position += velocity * deltaTime;  
rotation += angularVelocity * deltaTime;
```

- Play activa RuntimeBehaviorComponent
- Stop recupera posición, rotación y escala
- deltaTime elimina dependencia directa de FPS

Play/Stop cumple por estado, tiempo y restauración

Decisión técnica

La captura permanece en memoria porque su uso es temporal; el archivo de escena se reserva para persistencia permanente.

Criterio verificable

- Play inicia la lógica runtime
- Stop restaura el estado capturado
- La simulación usa unidades por segundo

Pregunta: ¿es Lua o C#?

No. El concepto de scripting se implementó con componentes runtime nativos en C++; no se integró una máquina virtual externa.

Audio separa el servicio global de cada fuente



- Sistema global: motor, listener y volumen maestro
- Componente: WAV, volumen, pitch, loop y distancias
- Transform del actor define la posición espacial

La posición relativa controla la experiencia sonora

```
difference = sourcePosition - listenerPosition;  
distance   = length(difference);
```

Atenuación

- Antes de Min Distance: intensidad alta
- Entre Min y Max: atenuación progresiva
- Después de Max: contribución mínima

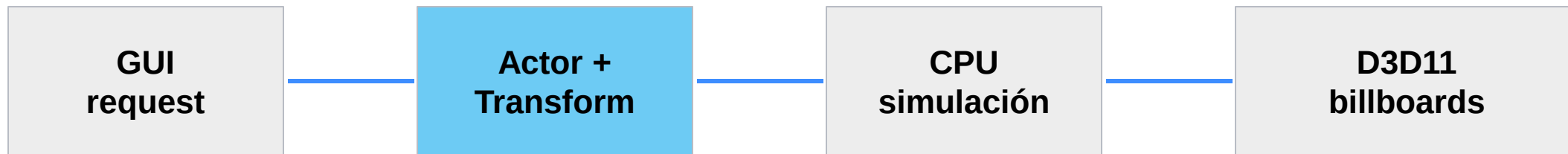
Persistencia

Ruta WAV · volumen · pitch · Spatial 3D · loop ·
mute · activación · distancias mínima y máxima

Respuesta técnica

Cumple fuente + listener + atenuación espacial. No pretende sustituir FMOD/Wwise ni incluye reverb u oclusión acústica.

Partículas integra ECS, CPU, GPU y editor



ParticleEmitterSettings concentra configuración serializable; cada Particle conserva solamente posición, velocidad, edad y lifetime.

```
Particle { position, velocity, age, lifetime }
```


Cuatro formas comparten la misma simulación

POINT

Origen + dispersión

BOX

Aleatorio dentro de extents

SPHERE

Volumen + dirección radial

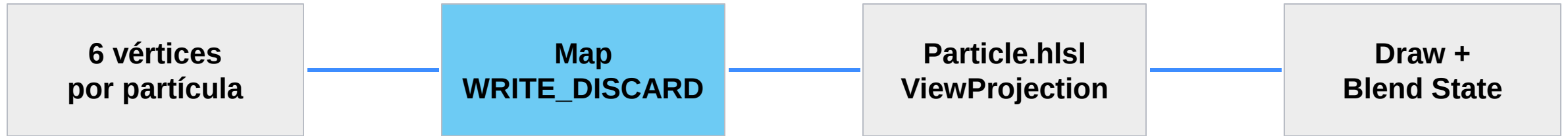
CONE

Base tangente + ángulo

```
speed    = lerp(minSpeed, maxSpeed, random);  
velocity = direction * speed;  
velocity += gravity * deltaTime;  
position += velocity * deltaTime;
```

Límite de seguridad: 1024 partículas vivas

Billboards dinámicos conectan CPU y DirectX 11



```
world = center + cameraRight * corner.x * size  
             + cameraUp    * corner.y * size;
```

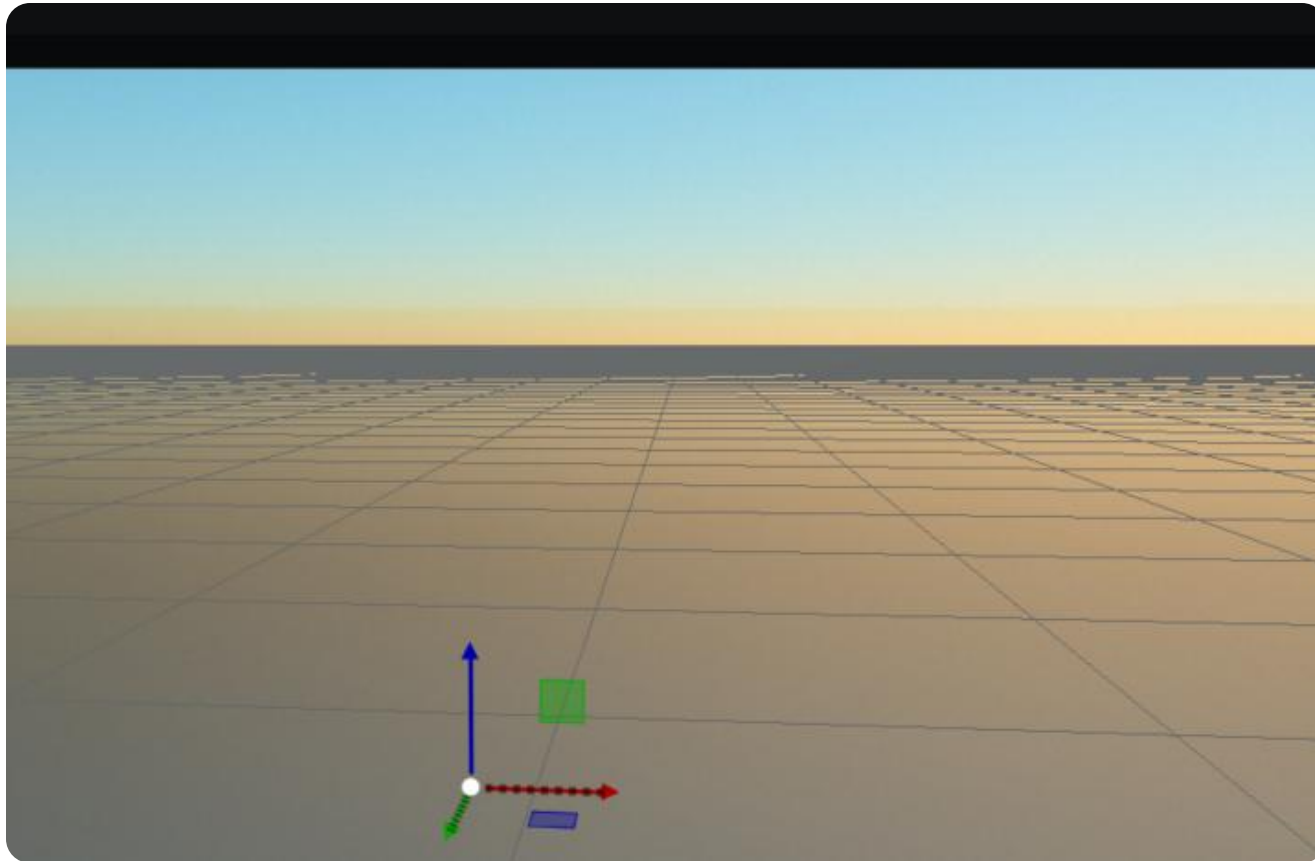
Additive

Fuego, energía y chispas

Alpha

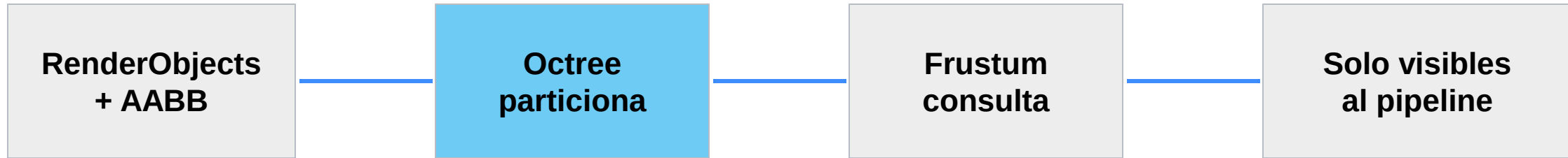
Humo, lluvia y polvo

Presets y escena v3 convierten el emisor en herramienta



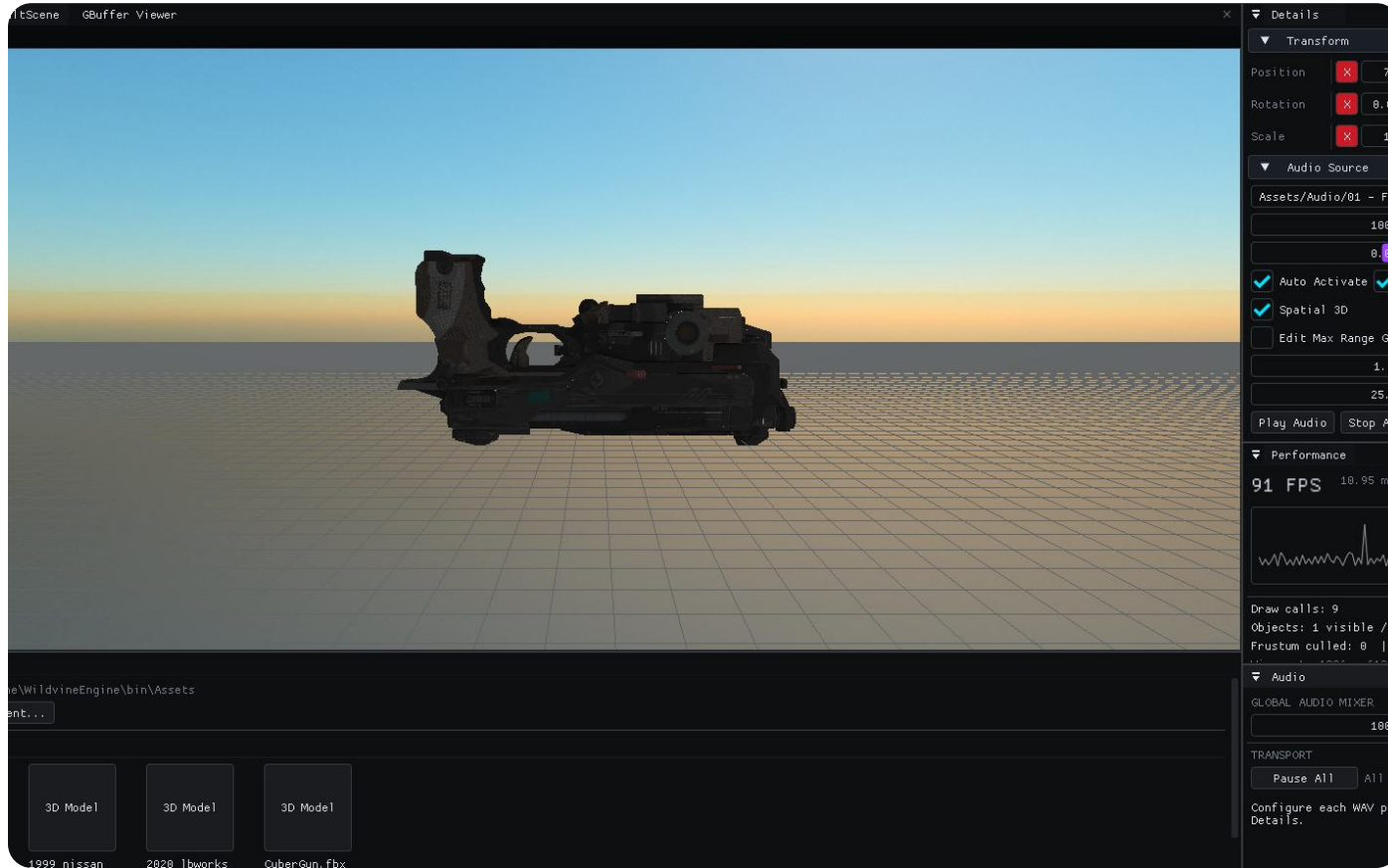
- Fire: Point + Continuous + Additive
- Smoke: Sphere + Alpha
- Sparks: Cone + Burst + gravedad
- Rain: Box + caída continua
- Escenas v1, v2 y v3 siguen cargando

Frustum y Octree eliminan trabajo antes del Draw



- Las ocho esquinas del AABB se transforman a mundo
- Un nodo fuera del frustum descarta toda su región
- El objeto descartado no llega a DrawIndexed

El profiler vuelve observable la optimización



Métricas

- FPS y frame time
- Draw calls
- Objetos totales y visibles
- Frustum culled
- Nodos del Octree

Cada herramienta tiene evidencia verificable

Herramienta	Núcleo técnico	Demostración	Cumple
Play/Stop	Captura TRS + deltaTime	Mover y restaurar	Sí
Audio 3D	Fuente + listener	Alejar cámara	Sí
Partículas	CPU + billboard + blend	Presets y Burst	Sí
Culling	AABB + Octree + frustum	Visible vs culled	Sí

El alcance seleccionado está completo; no se afirma equivalencia con middleware o motores comerciales.

Preguntas técnicas que conviene responder con precisión

¿Lua/C#?

Comportamientos nativos C++; sin VM externa

¿Audio profesional?

Núcleo 3D sobre DirectXTK; no FMOD/Wwise

¿Partículas GPU?

CPU hasta 1024; GPU sería la siguiente escala

¿Curvas avanzadas?

Interpolación lineal por vida normalizada

¿Occlusion culling?

No; frustum + partición espacial

Cuatro sistemas completos, conectados al mismo frame

Runtime · sonido · VFX · optimización

Código propio: 71/71 headers documentados · Doxygen 0 warnings · compilación 0 errores